

Working with GitHub

How your website follows you from Mac to Mac: set up once, then commit, sync and check.

You will sit at a different Mac almost every session, and each Mac keeps its own separate home folder for your login. Anything you leave on one machine's Desktop stays on that machine. So your website does not live on any lab Mac: it lives on **GitHub**, and each Mac is just a place you work on it. You pull your site down at the start of a session, and you push your changes up before you leave. Once that habit sticks, losing work becomes almost impossible, and you get a complete history of every version of your site for free.

Never put your website folder inside OneDrive. OneDrive is fine for essays and files you upload through the browser, but it silently corrupts the hidden change-tracking data that git keeps inside your website folder. Your website folder goes on the Desktop; GitHub is its backup, not OneDrive.

This guide has four parts. You only ever do Part 1 once. Part 2 is the start-of-session routine for a machine you have not used before, and Part 3 is the rhythm of every working session. Part 4 puts your site on the live web, once, and from then on your syncs update the internet.

PART 1

The first time

Create your site and put it on GitHub.
Done once, in Week 1.

PART 2

Sitting at a new Mac

Get your site back and re-introduce
yourself to git.

PART 3

Every time you work

Edit, commit, sync, and check before you
leave.

PART 4

Going live

Turn on GitHub Pages and get a real web
address. Done once.

1 The first time

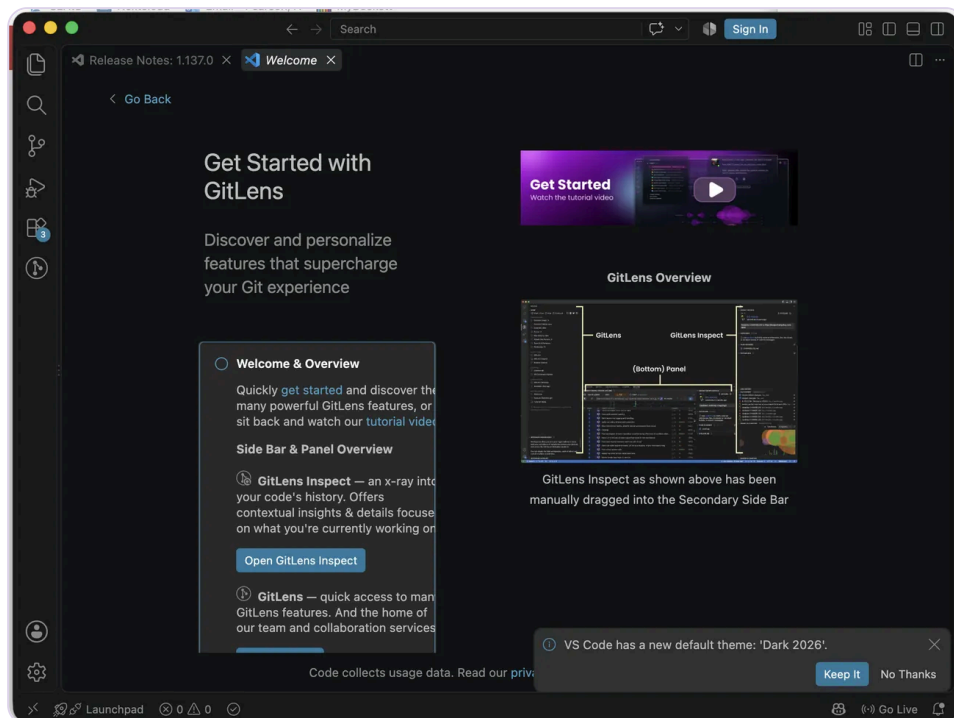
ONCE, IN WEEK 1

By the end of this part you will have a website folder on the Desktop, tracked by git, published to your own GitHub account, and visible at github.com. Work through the steps in order: two of them exist purely to stop errors appearing later.

Doing this in the Week 1 session? There you start from the downloaded starter project, not an empty folder. The steps are identical with two differences: wherever this guide says **my-website**, your folder and repository are called **interaction-design**, and you skip step 1.4 because the starter already has an **index.html**. The screenshots show the empty-folder version.

1.1 Clear the clutter

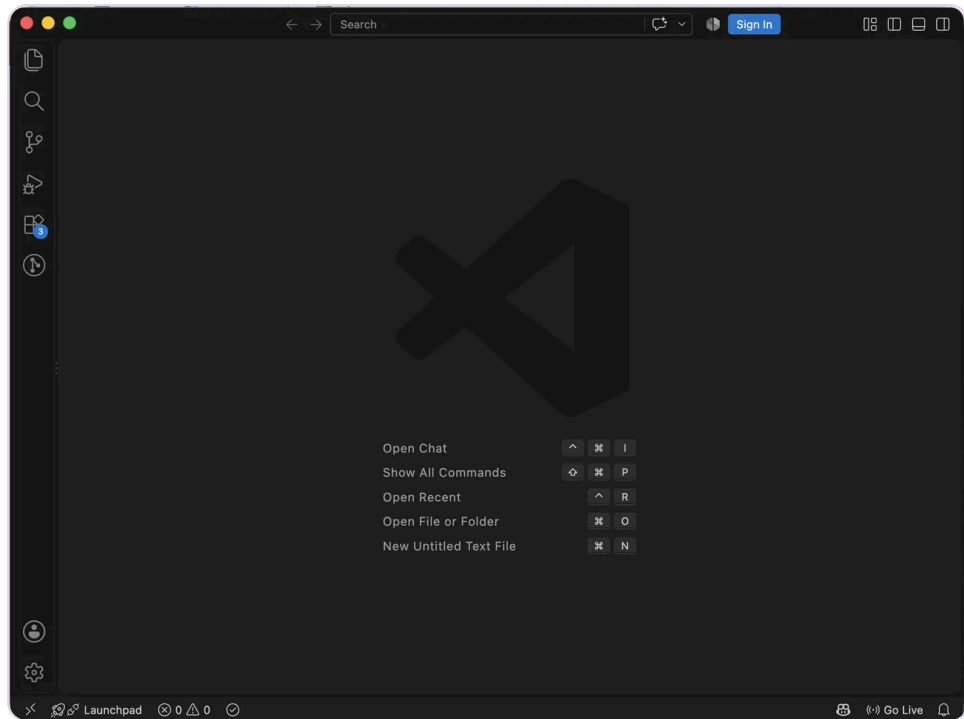
The first time VS Code opens on a lab Mac it throws several things at you at once. None of them matter.



First launch: a GitLens tab, a Release Notes tab and a theme popup, all at once. All ignorable.

1. Click **No Thanks** on the theme popup (or Keep It, it makes no difference).
2. Close every open tab: Release Notes, Welcome, GitLens.

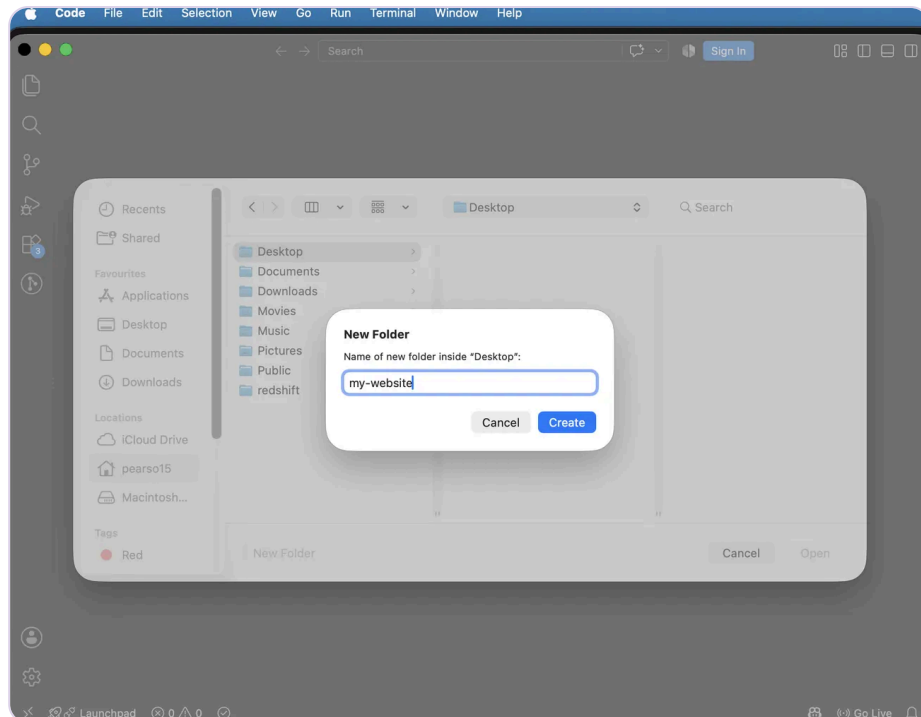
Ignore everything GitLens asks for. GitLens is an extension installed on the lab Macs. It will repeatedly invite you to create an account, sign in, or try Launchpad Pro. You never need any of it. The only account this module uses is your GitHub account, and the only sign-in happens in the web browser later in this guide. Also ignore the blue **Sign In** button at the top right of VS Code itself: that is a Microsoft account sign-in, not GitHub.



What you want before you start: an empty editor, no tabs, no popups.

1.2 Make your folder

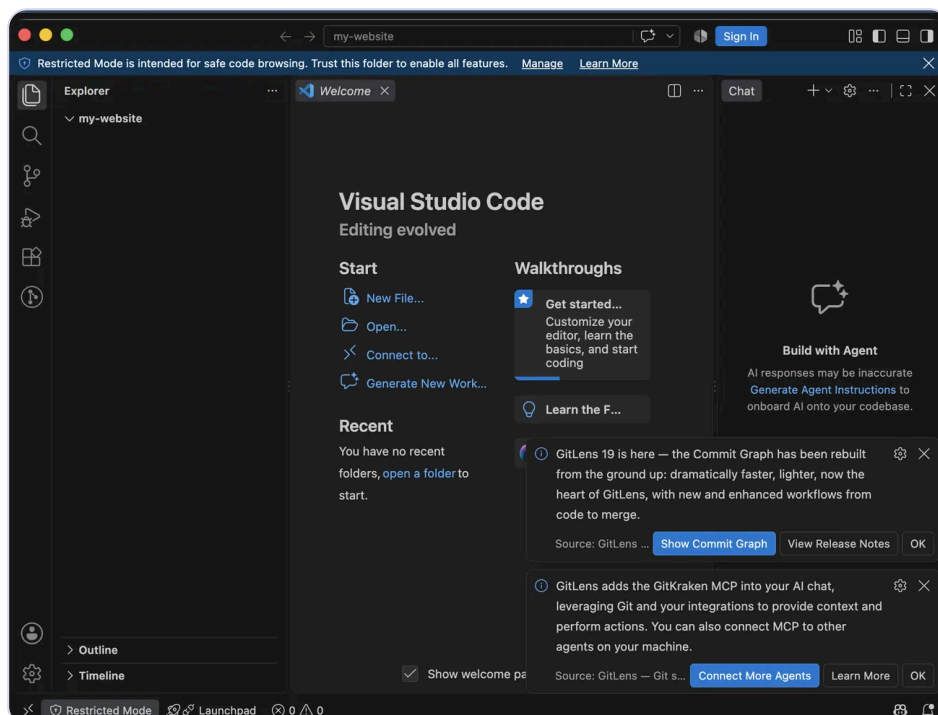
1. Go to **File** → **Open Folder...**
2. In the dialog, click **Desktop** in the sidebar.
3. Click **New Folder**, name it **my-website** (lower case, hyphen, no spaces), and click **Create**.
4. With the new folder selected, click **Open**.



Name it exactly **my-website** . Lower case with hyphens is the convention for anything that ends up in a web address.

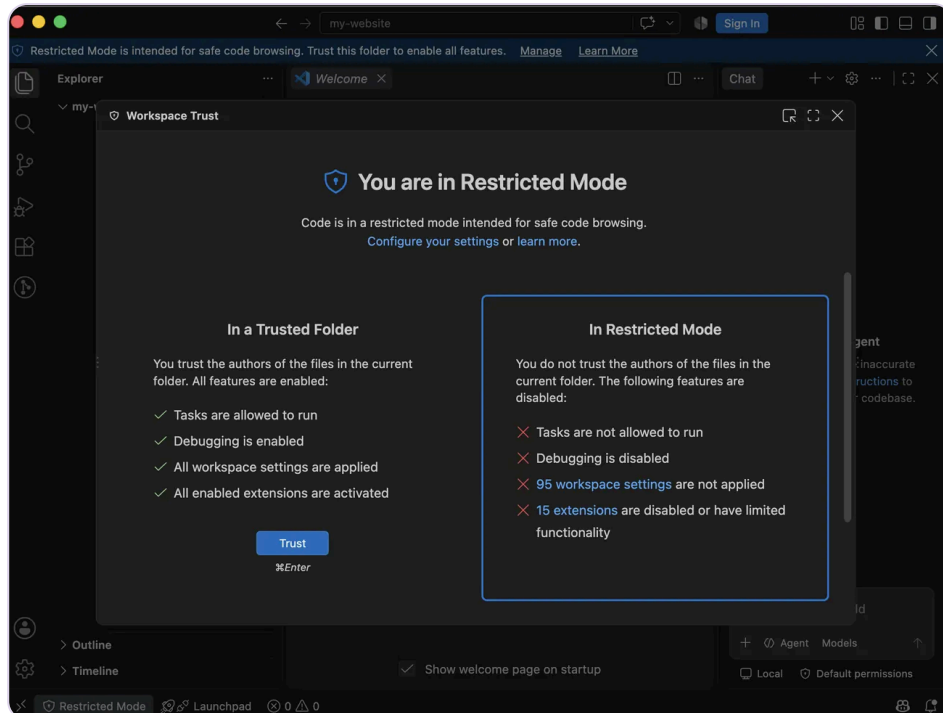
1.3 Trust the folder

VS Code opens the folder in Restricted Mode, a safety setting for code you did not write. While the blue banner is showing, git is switched off, so this step is not optional.



The blue banner and the "Restricted Mode" label in the bottom status bar. More GitLens popups appear here too; click OK and move on.

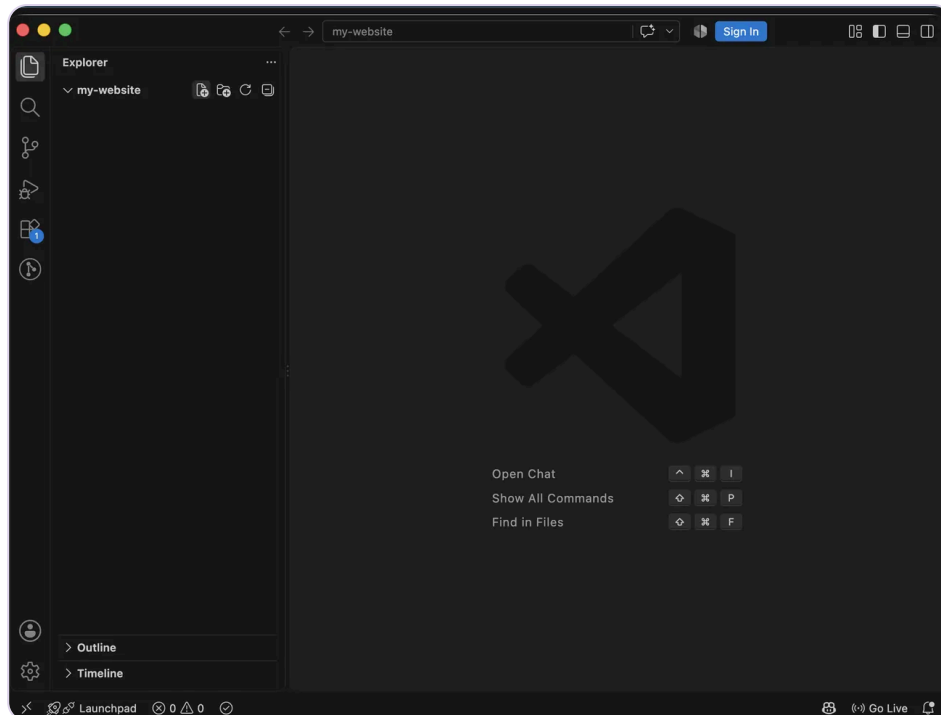
1. Click **Manage** in the blue banner.
2. On the screen that opens, click the big blue **Trust** button.
3. Close the Workspace Trust tab. If a file chooser opens at any point here, you have clicked **Add Folder** by mistake: click **Cancel**.



Click **Trust**. Success check: the banner disappears and "Restricted Mode" vanishes from the status bar.

1.4 Make your first page

1. In the Explorer panel on the left, click the **New File** icon next to the folder name.
2. Type **index.html** and press **Return**.
3. Type **!** then press **Tab** to expand the HTML boilerplate, and put a heading in the body.
4. Save with **⌘S**. The dot on the file's tab turns into an **x** when it is saved; a dot means unsaved changes.



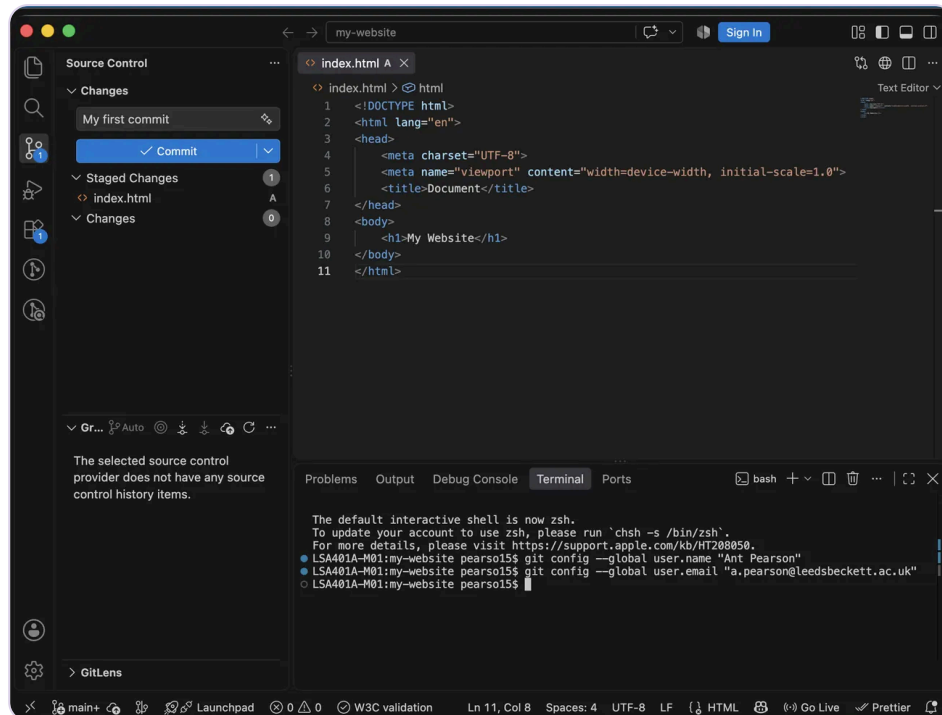
A saved index.html. Git can only record what has been saved, so **%S** before every commit.

1.5 Tell git who you are

Every change you record is stamped with a name and an email address. The lab Macs do not know who you are, so you tell git once per machine. Skipping this is the single most common cause of a confusing error later.

1. Go to **Terminal** → **New Terminal**. A panel opens along the bottom of the window. (If the menu is greyed out, a dialog box is open somewhere: close it and try again.)
2. Type these two lines, pressing **Return** after each. Use your own name, and the email address you used for your GitHub account:

```
git config --global user.name "Your Name"
git config --global user.email "yourname@example.com"
```



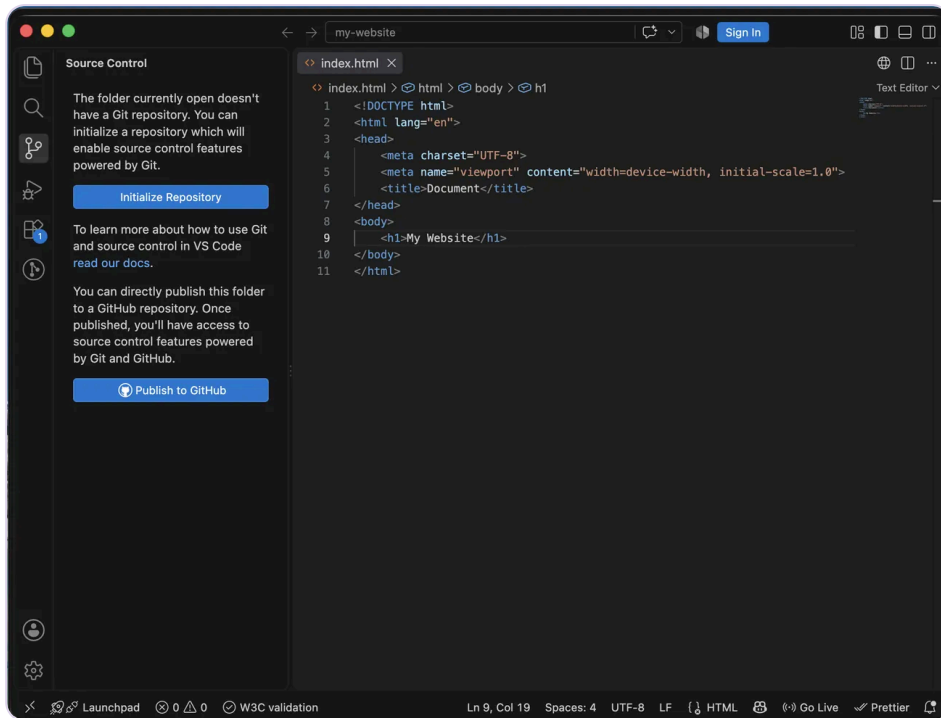
No news is good news: the commands print nothing when they work.

Why the email matters: GitHub links your saved changes to your account by matching this email address. Use the one you signed up to GitHub with and your commits show up as yours.

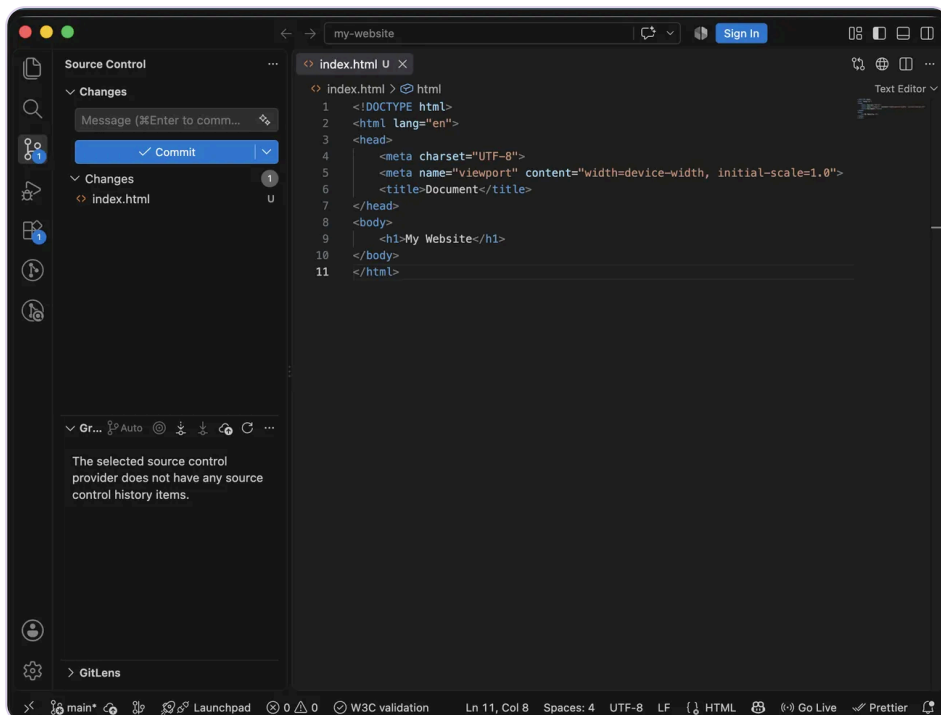
1.6 Your first commit

A commit is a snapshot of your folder with a short label saying what changed. It is the basic unit of change-tracking, and you will make many of them every session.

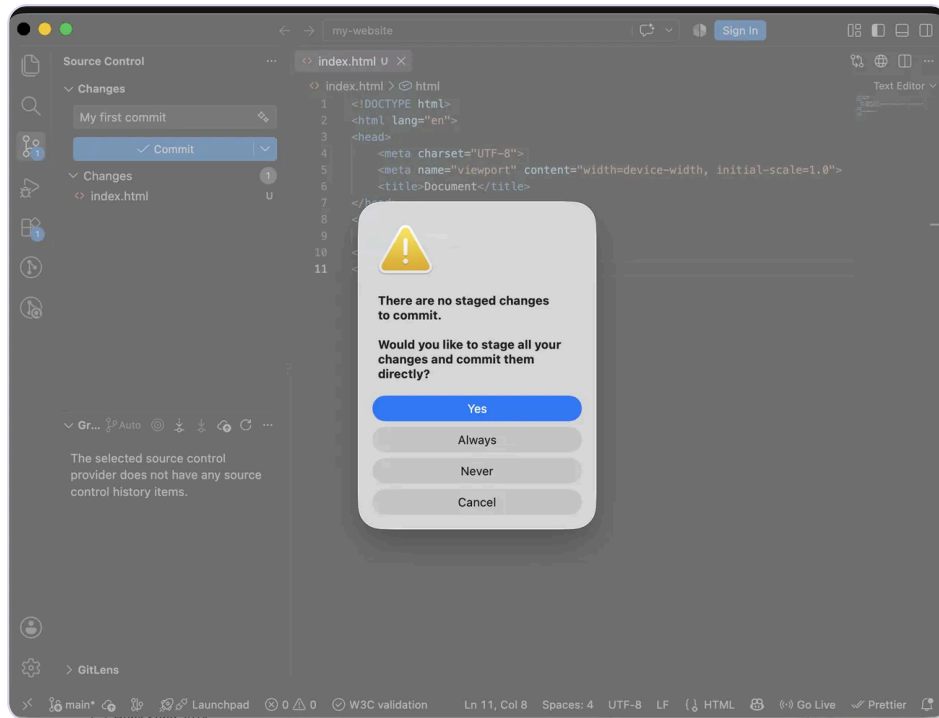
1. Click the **Source Control** icon in the left bar (third one down, the branching lines).
2. Click **Initialize Repository**. This is a once-per-website action: it switches git on for this folder.
3. Your file appears under Changes with a **U** (untracked, meaning new). Type a short message in the box, something like **My first commit**, and click **Commit**.
4. A box asks whether to stage all changes and commit them directly. Click **Yes**. (Staging is choosing which changes go in the snapshot; "all of them" is right for this module.)



Before: two buttons. **Initialize Repository** is the one you want.

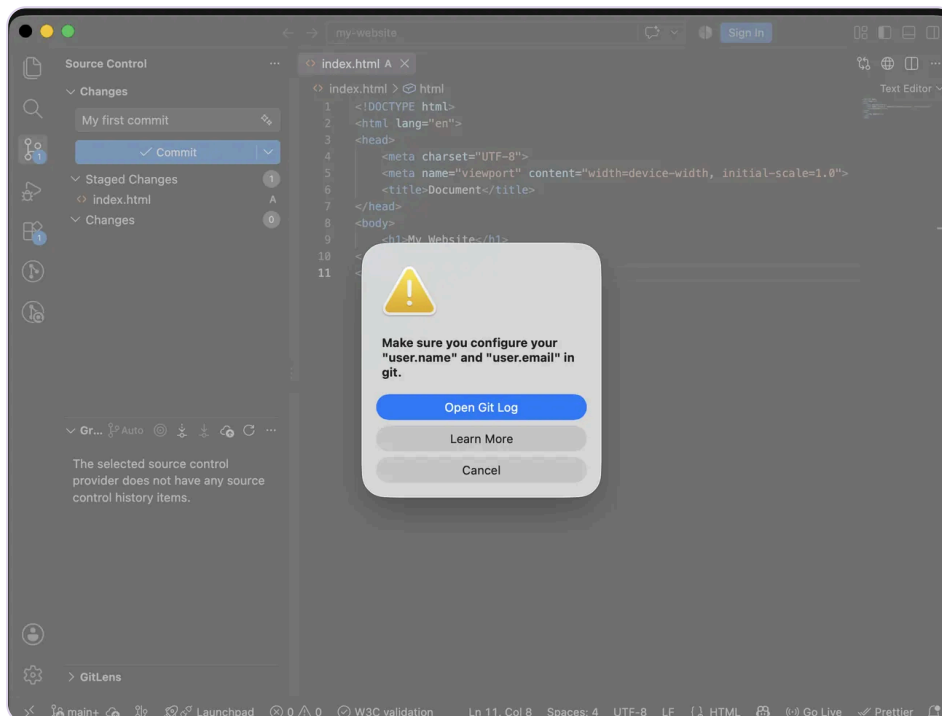


After: your file listed under Changes, a message box, and a Commit button. The status bar now shows a branch called **main**.



Click **Yes** each time this appears. It will keep appearing on new machines, so get used to it rather than hiding it.

If you see "Make sure you configure your user.name and user.email in git", you skipped step 1.5. None of the buttons on this dialog help: click **Cancel**, do step 1.5, then click **Commit** again.

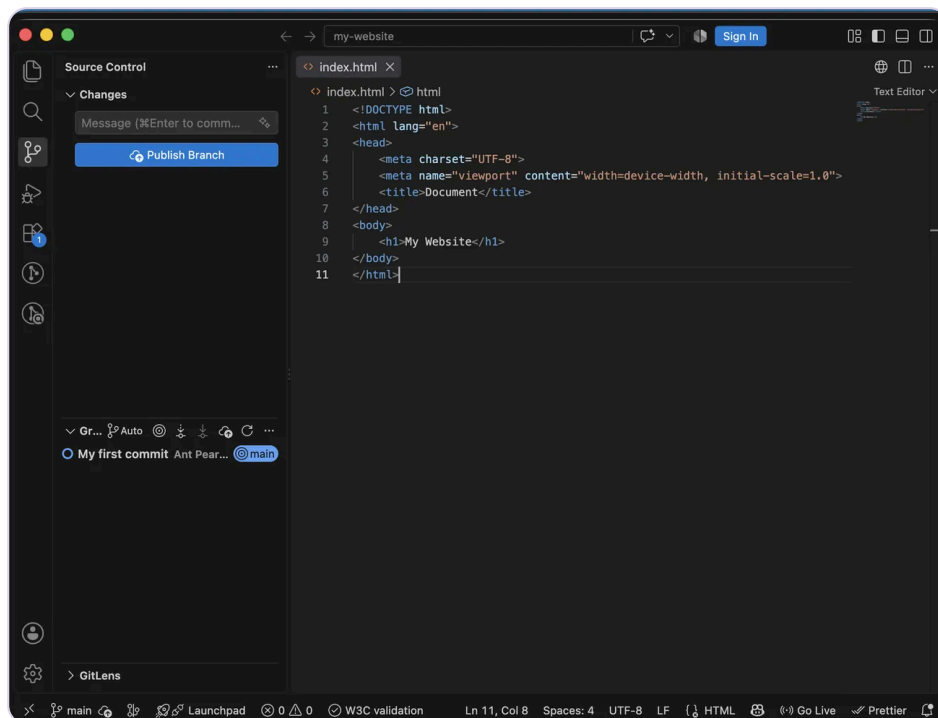


The unhelpful dialog in question. Cancel, run the two lines from step 1.5, commit again.

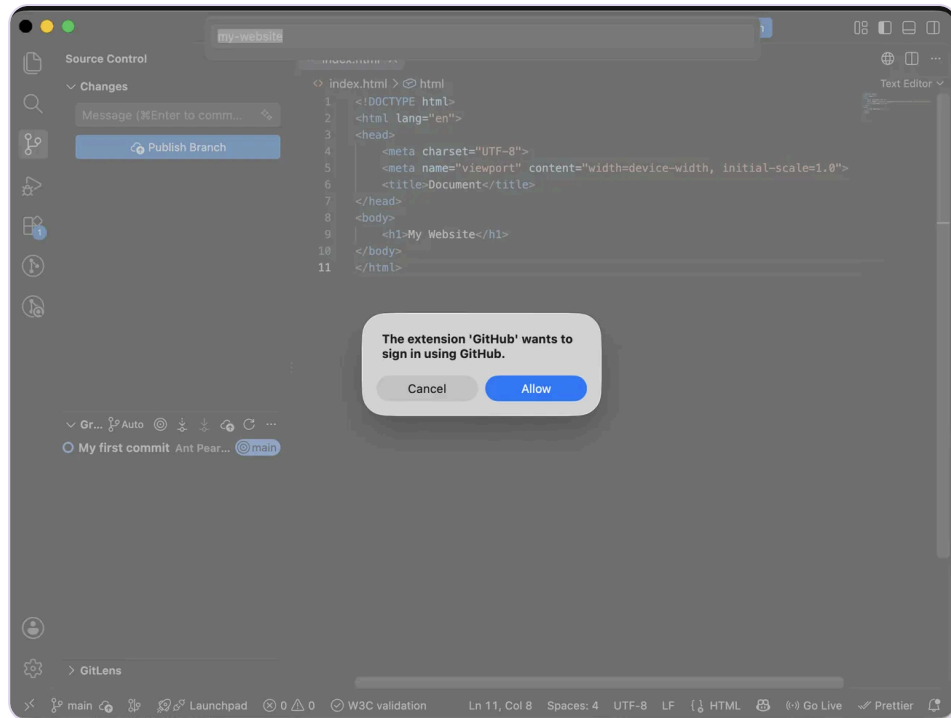
1.7 Publish to GitHub

So far the snapshot only exists on this Mac. Publishing creates a copy of your website on GitHub, under your account, and connects the two. This is the longest step because GitHub needs to check who you are, which happens in the web browser.

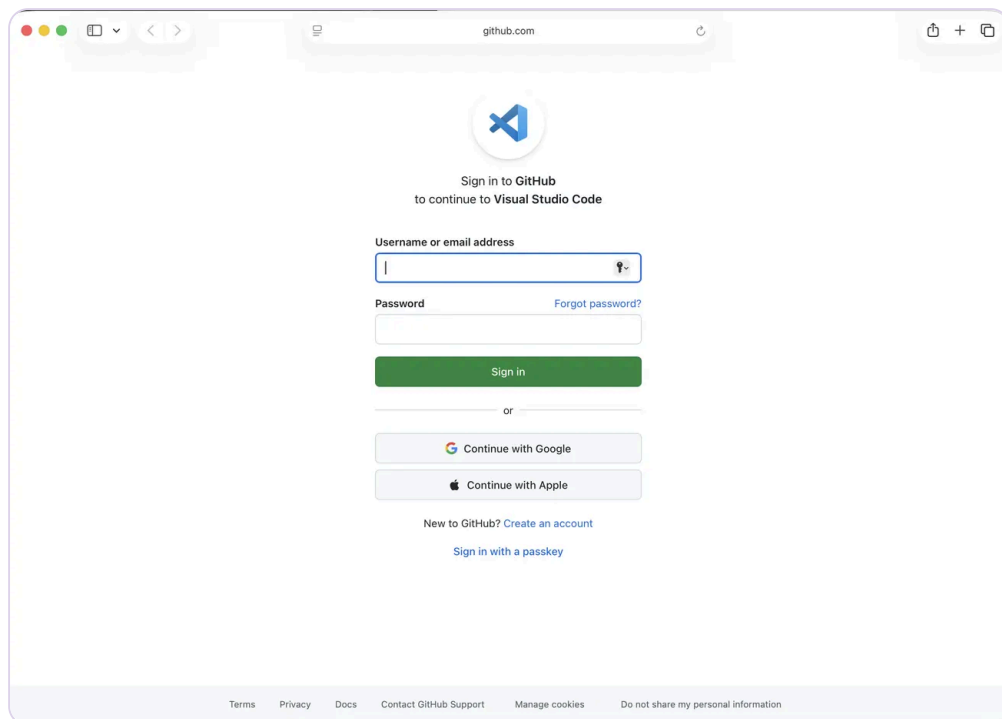
1. After your first commit, the button changes to **Publish Branch**. Click it.
2. A dialog says the GitHub extension wants to sign in. Click **Allow**. Your web browser opens on github.com.
3. Sign in with your GitHub username and password, then complete two-factor authentication with your phone.
4. On the Authorize Visual Studio Code page, check it says your username, then click **Continue** or the green authorise button.
5. When the browser says you have successfully signed in, go back to VS Code.



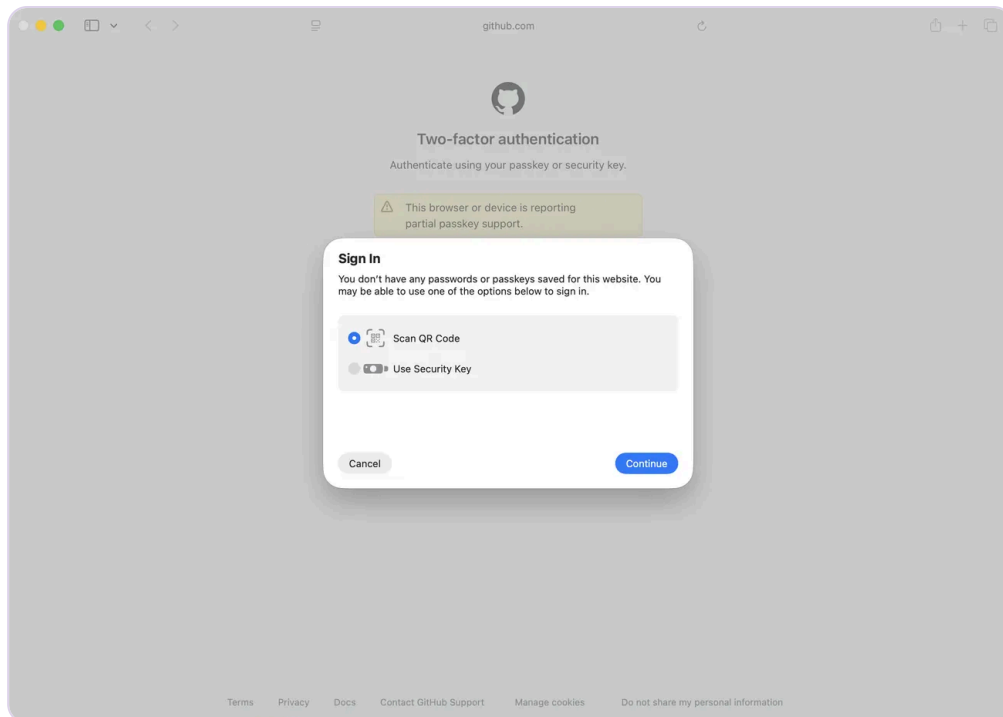
The commit has landed (see it in the Graph at the bottom left) and the button now reads **Publish Branch**.



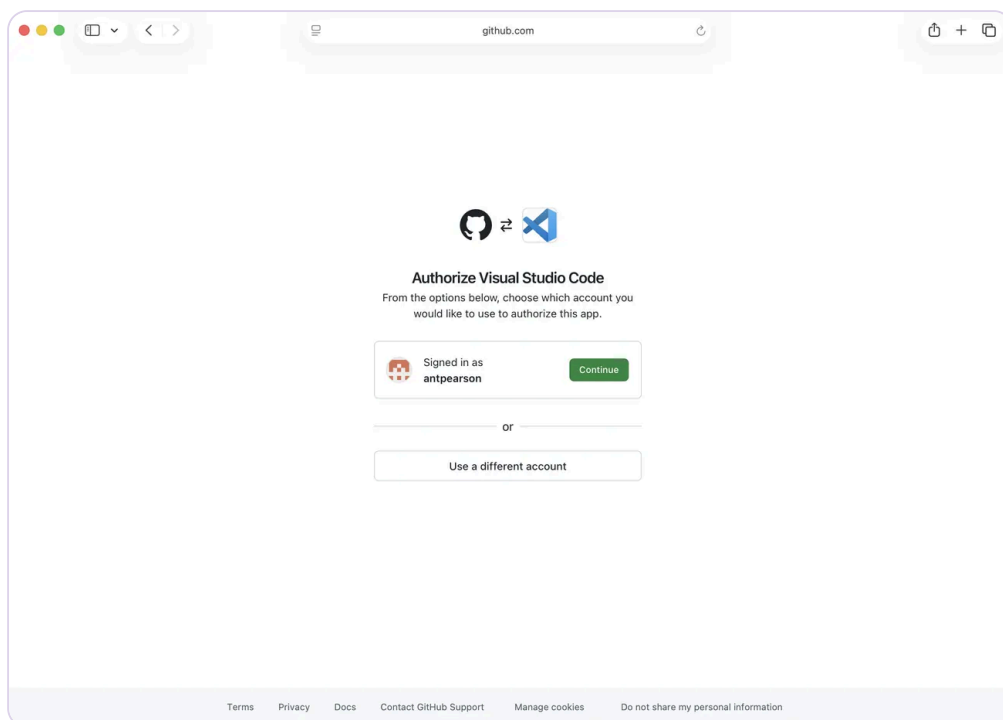
Click **Allow**. This is the one sign-in that is real: it hands over to github.com in the browser.



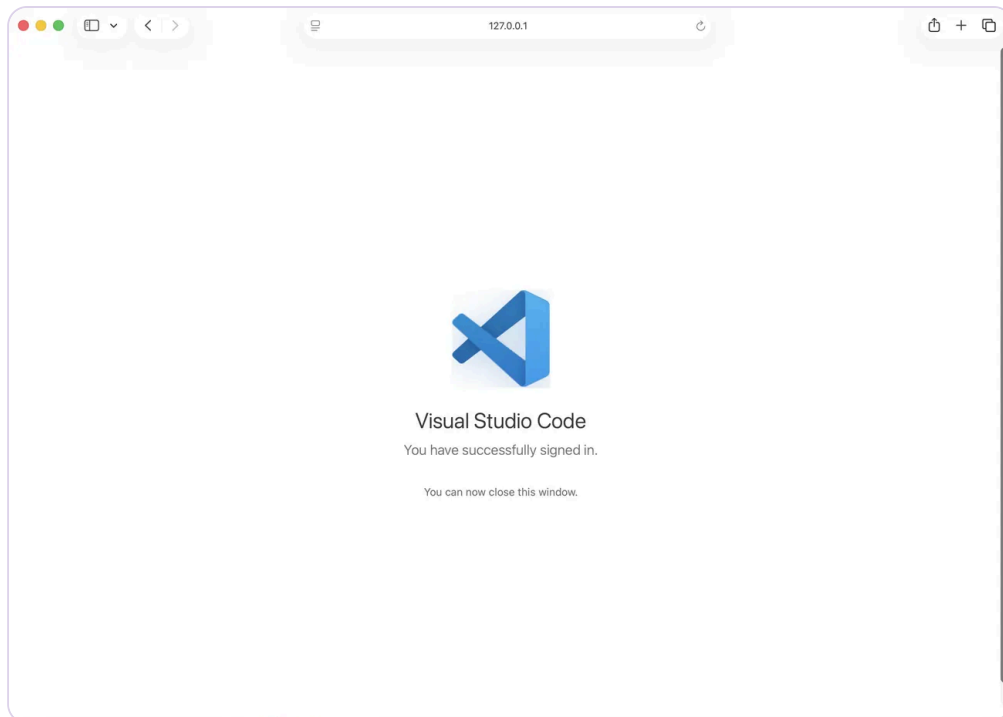
The browser takes over. Check the address bar says **github.com** before typing your password.



Two-factor authentication looks different depending on how your account is set up: a code from the GitHub app, a text, or scanning a QR code with your phone. On a shared machine, never tick anything offering to remember this browser or device.



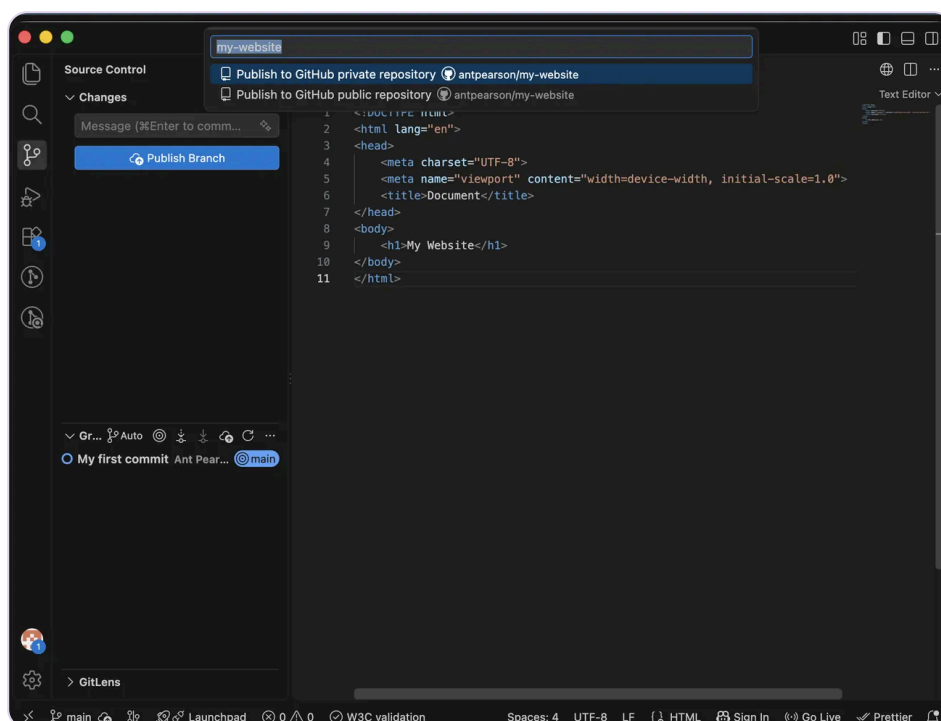
Check the username is yours, then continue.



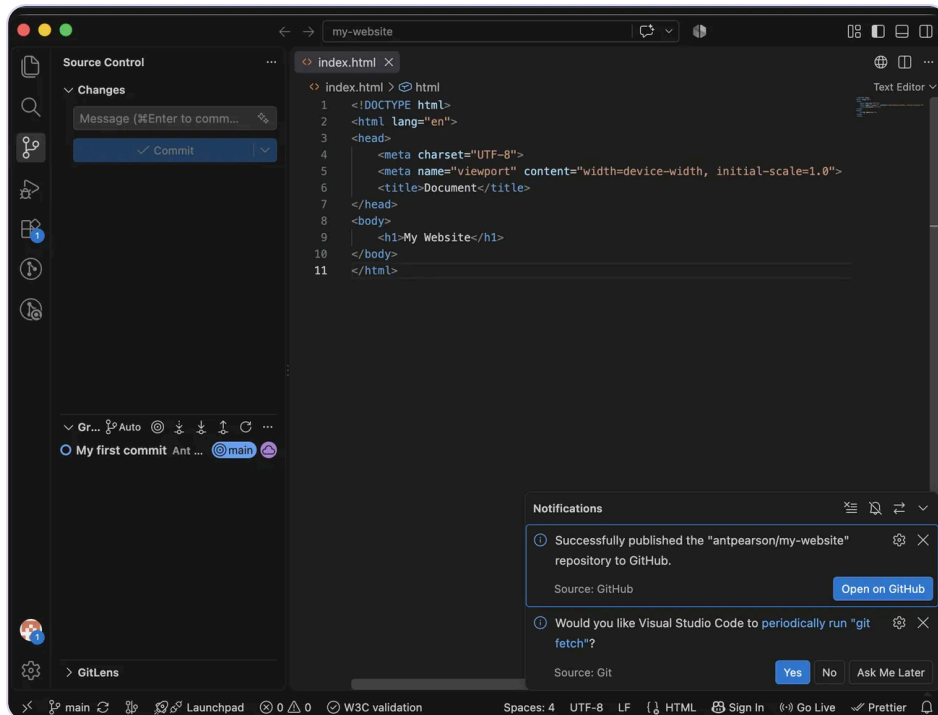
Done in the browser. Back to VS Code.

Back in VS Code, a box drops down from the top asking where to publish.

The top option is a private repository, and it is selected by default. Do not press Return. Click the second option, Publish to GitHub public repository. Your site needs to be public to go on the web later, and everything in this folder is coursework, not secrets.



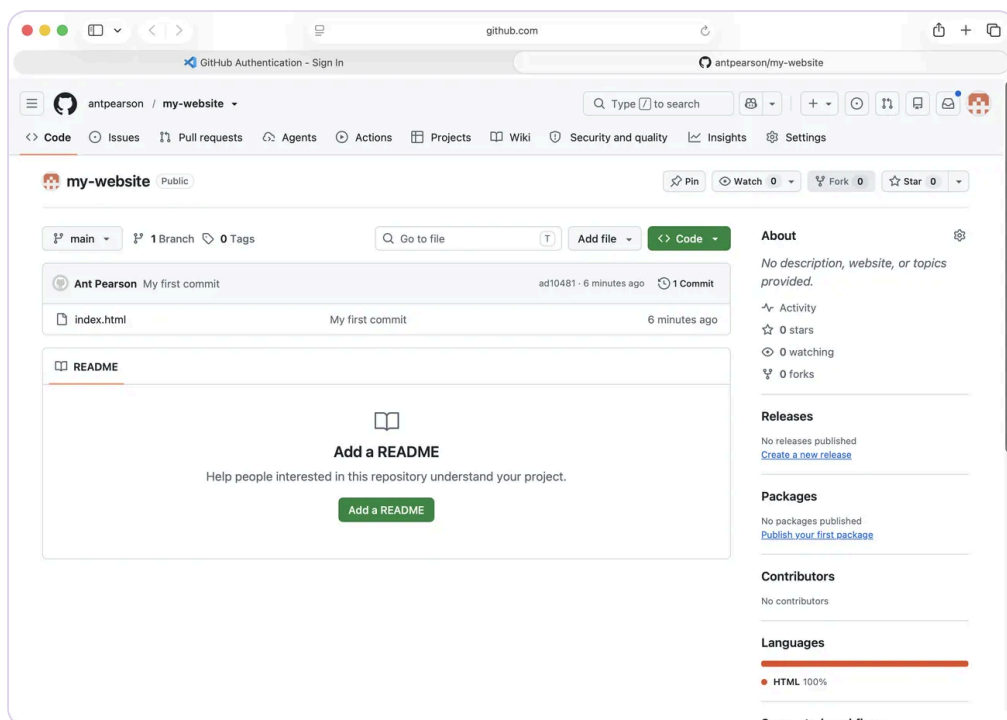
The trap: private is pre-selected. You want **public**, the second row.



The confirmation, bottom right. If another popup asks whether to periodically run git fetch, click **Yes**.

1.8 See it on github.com

Click **Open on GitHub** in the notification, or go to `github.com` and open your profile. Your repository is there, with your file and your commit message next to it. This page is your proof that the work is safely off the lab Mac, and you will check it at the end of every session from now on.

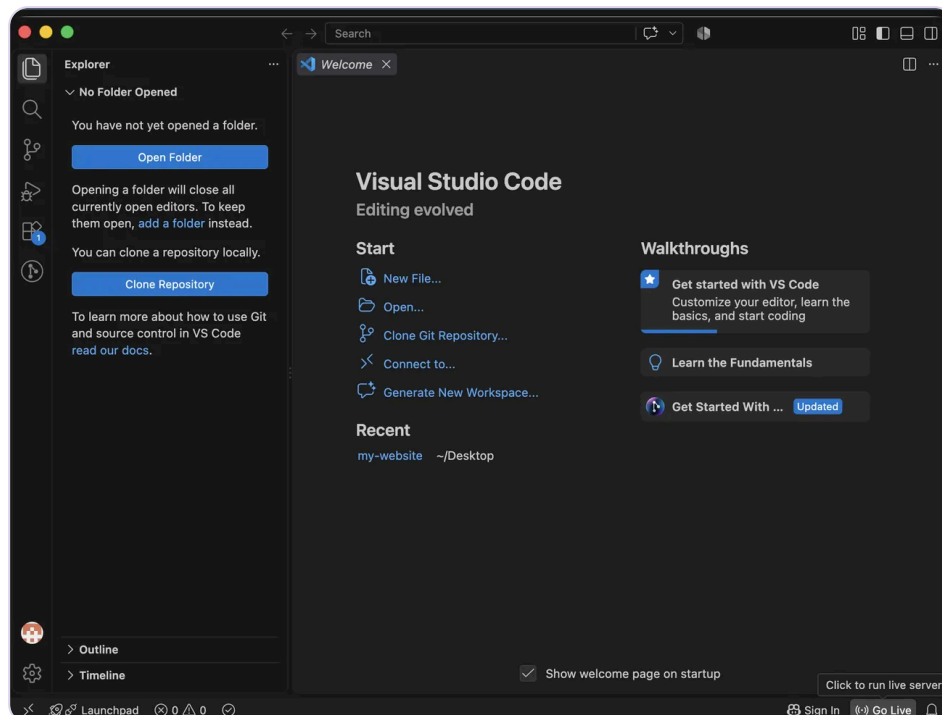


Your website, living on GitHub. The commit message and time appear next to each file.

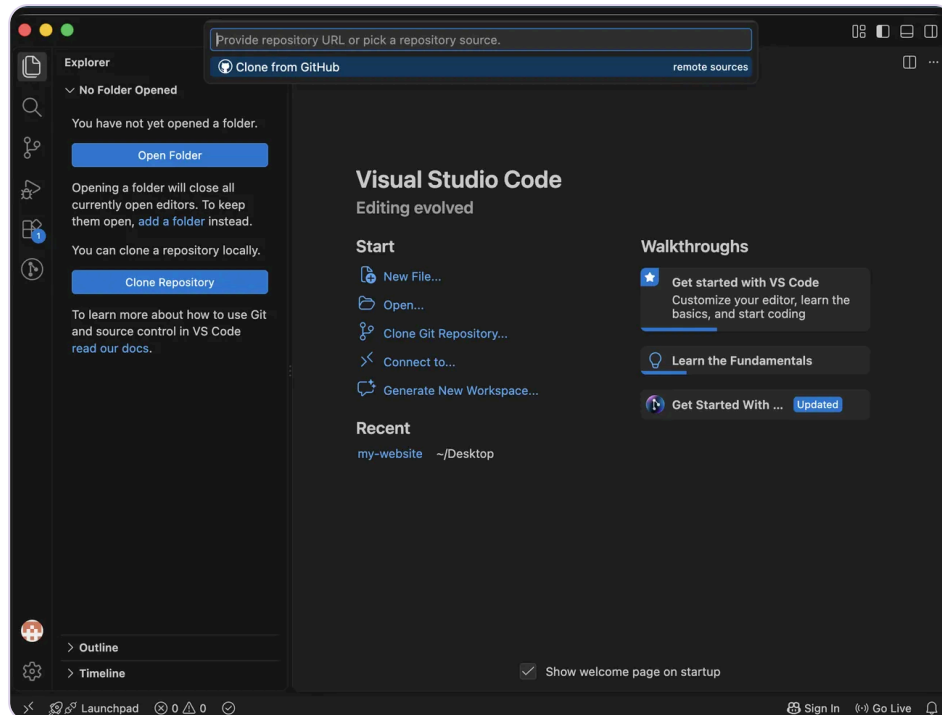
Next session you will probably be at a different machine, and your Desktop folder from last week is not on it. That is by design: your site lives on GitHub, so you **clone** it, which means downloading a full working copy, history and all. There are only two steps, plus one about what to ignore.

2.1 Clone your website

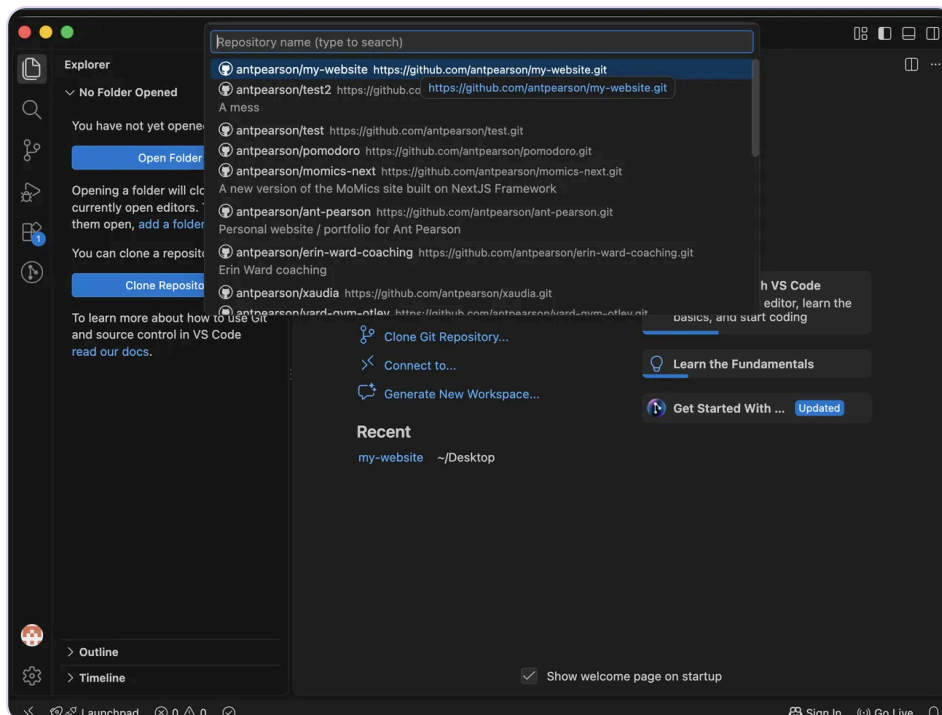
1. Open VS Code and clear any clutter (step 1.1).
2. On the Welcome tab click **Clone Git Repository...** (or the **Clone Repository** button in the Explorer panel, they do the same thing).
3. In the box that drops down from the top, click **Clone from GitHub**.
4. If the browser opens asking you to sign in to GitHub, work through it exactly as in step 1.7, then come back.
5. Pick **yourusername/my-website** from the list of your repositories.
6. In the folder chooser, select **Desktop** and click **Select as Repository Destination**.
7. When asked, click **Open**.



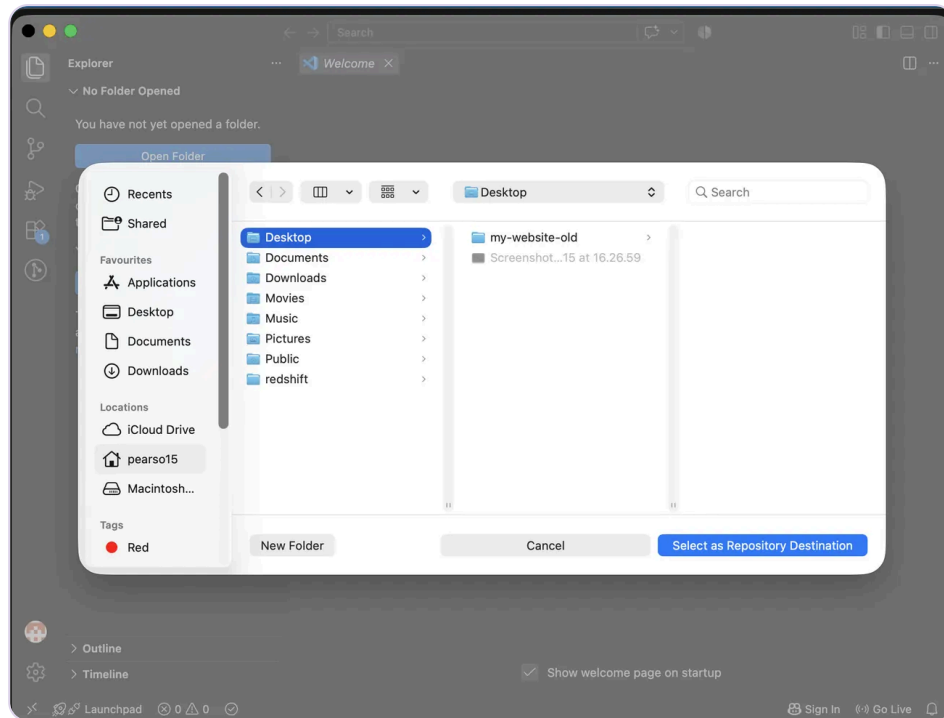
Two routes to the same place. Ignore the **Recent** list on the Welcome tab: on a new machine it is empty or points at folders that are not there.



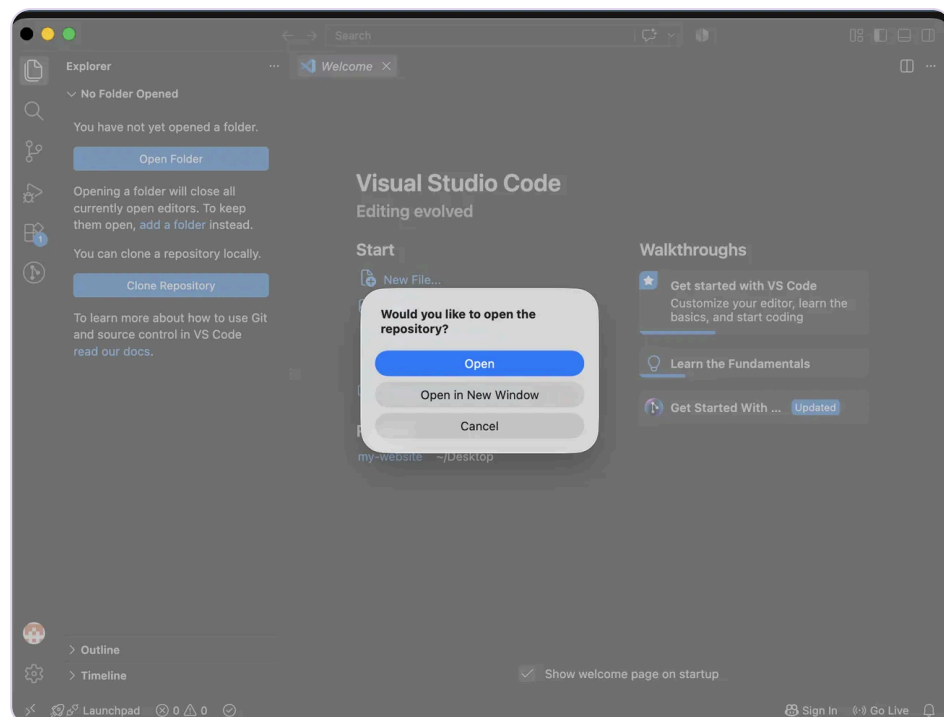
Click **Clone from GitHub**. No URL typing needed.



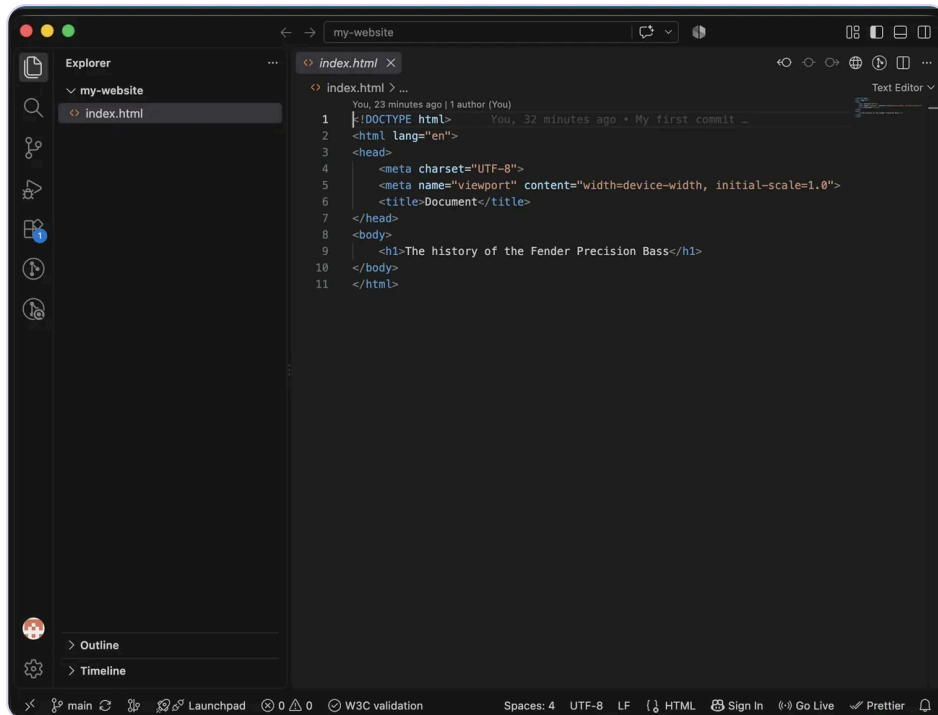
Your repositories. Pick **my-website**.



Desktop, then **Select as Repository Destination**.



Click **Open**.



Everything back, exactly as you left it last session. A folder VS Code cloned itself is trusted automatically, so no Restricted Mode banner this time.

2.2 Tell git who you are, again

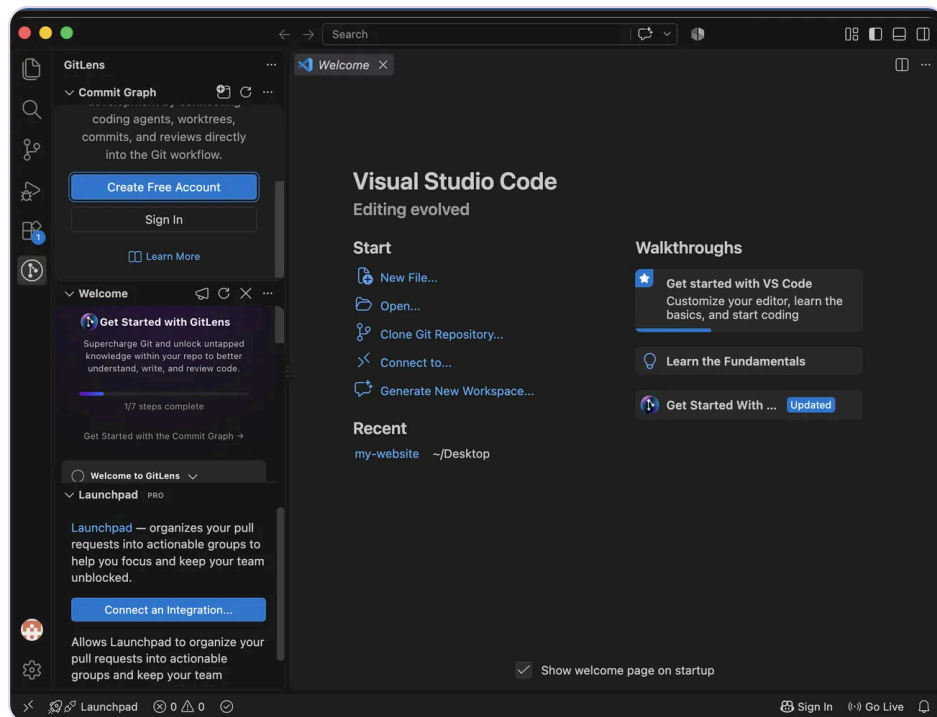
Your name and email from step 1.5 were saved on last week's machine, not this one. Run the same two lines in a new terminal (**Terminal** → **New Terminal**):

```
git config --global user.name "Your Name"
git config --global user.email "yourname@example.com"
```

Do it now, before you start working, and the confusing dialog from step 1.6 never appears.

2.3 Things to ignore on a fresh machine

- **GitLens** will fill the sidebar with Create Free Account, Sign In and Launchpad Pro. None of it is needed; close it and carry on.
- **The Recent list** on the Welcome tab remembers folders from whoever used the machine's login state last, or from machines you are no longer at. Always clone; never trust Recent.
- **The blue Sign In button** top right is a Microsoft account. Not needed.



GitLens making itself at home in an empty window. Everything in the left panel here can be ignored.

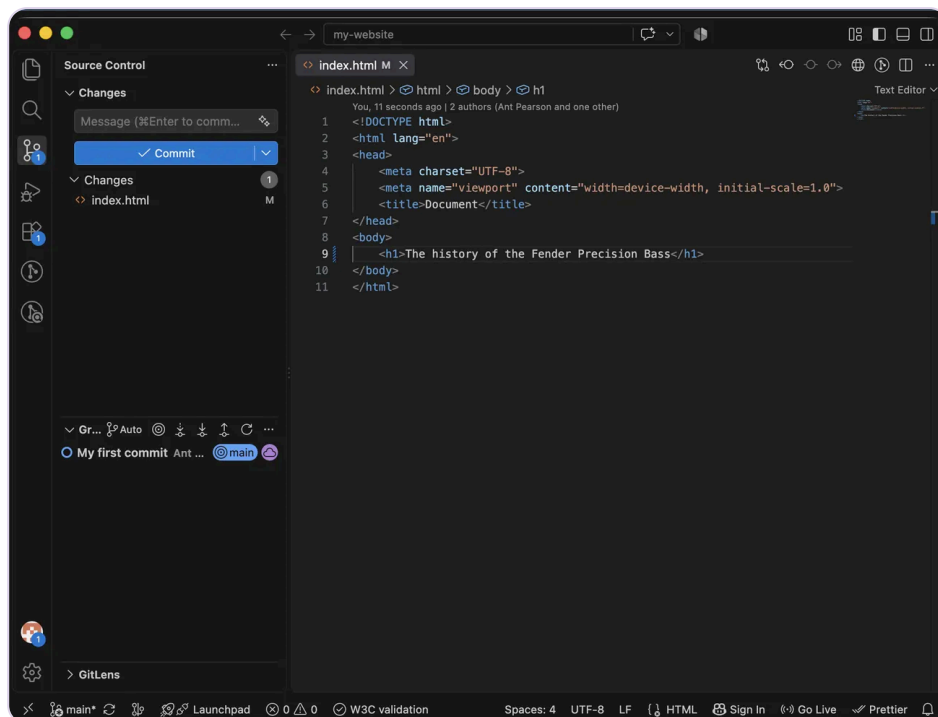
3 Every time you work

THE SESSION RHYTHM

This is the habit the whole guide exists for. You edit, you commit whenever you finish a meaningful chunk, and you sync before you leave. Commit little and often: a commit that says **Added the hero image** is far more useful to you later than one that says **everything from today**.

3.1 Edit and save

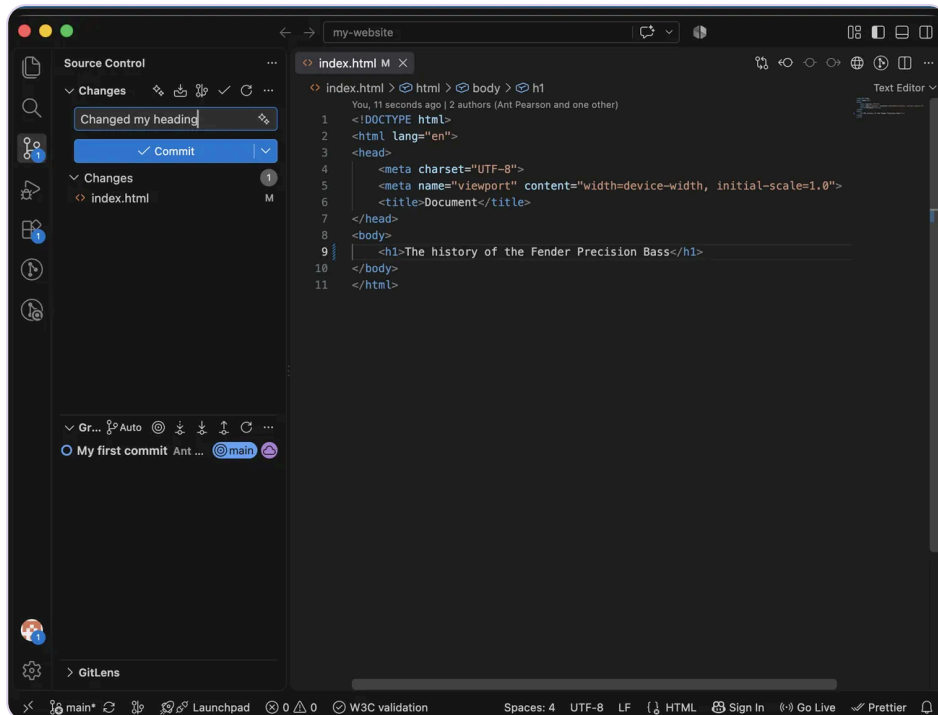
Work on your site as normal. As soon as you save a change, Source Control notices: the file appears under Changes with an **M** (modified), and a number appears on the Source Control icon.



One saved change, one **M**. Unsaved changes do not count, so **%S** first.

3.2 Commit

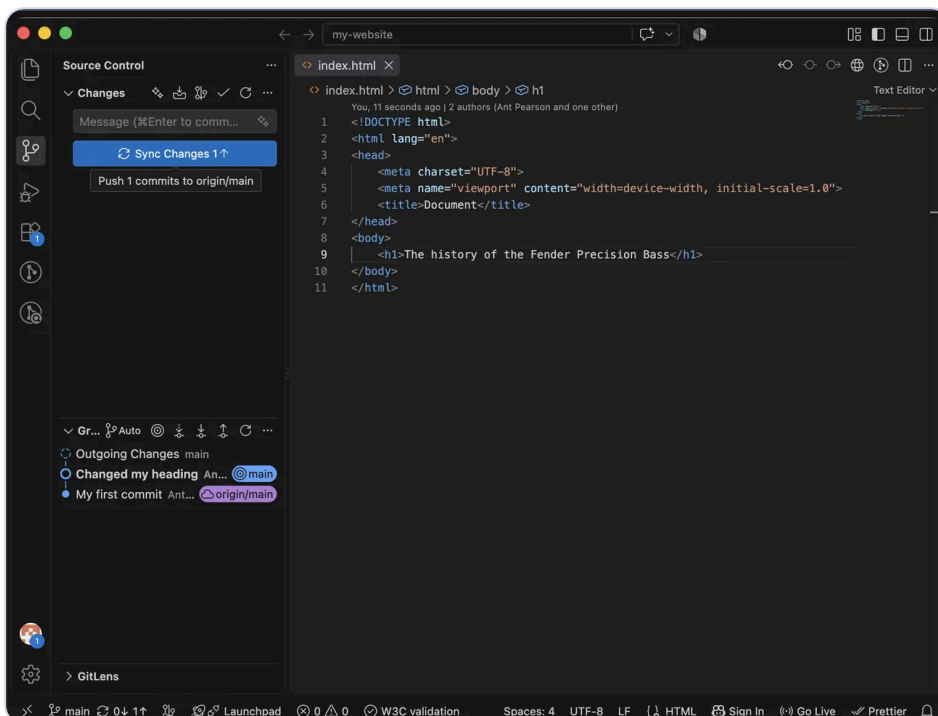
Type a short message describing what you changed, click **Commit**, and click **Yes** if the staging box appears.



Write the message before you click Commit. Say what changed, briefly, in the present or past tense, your choice, just be consistent.

3.3 Sync

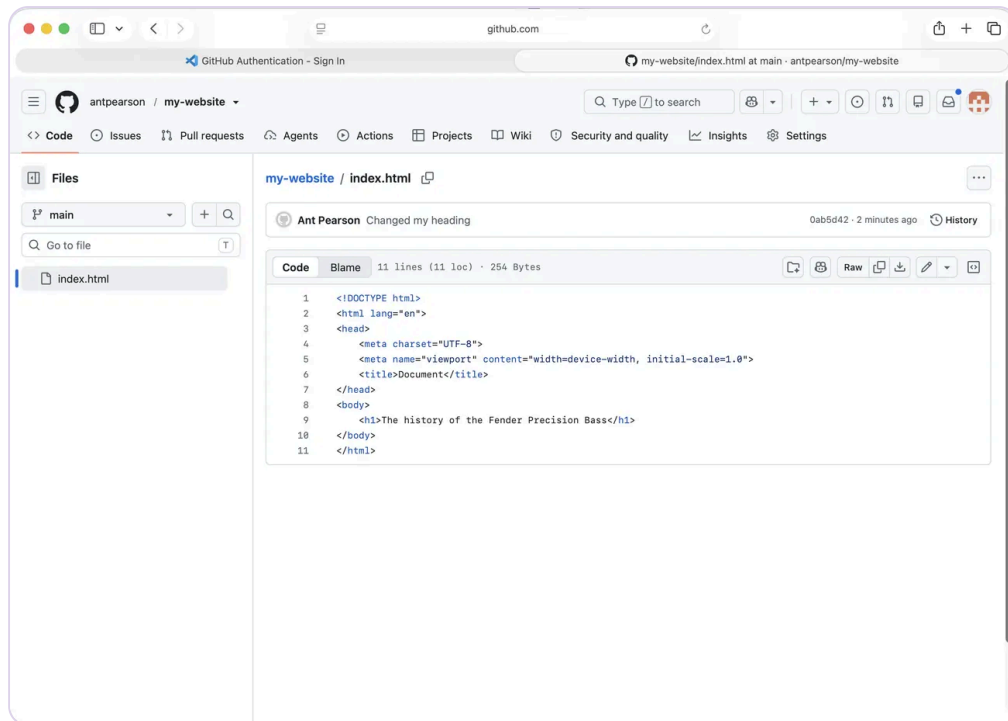
Committing records the snapshot on this Mac only. The button now reads **Sync Changes** with an arrow and a number: that is how many commits exist only on this machine. Click it, and they are pushed up to GitHub.



The Graph tells the story: the new commit is on **main** on this machine, while GitHub (**origin/main**) is still one behind. Sync closes the gap.

3.4 Check before you leave

Open your repository on github.com and refresh. Your latest commit message should be showing at the top. If it is, your work is off this machine and next week's clone will include it. If it is not, you have committed but not synced: go back to VS Code and click **Sync Changes**. Once your site is live (Part 4), your public web page also updates itself a minute or two after every sync.



The proof: the change and its message, live on GitHub.

End of session, every session

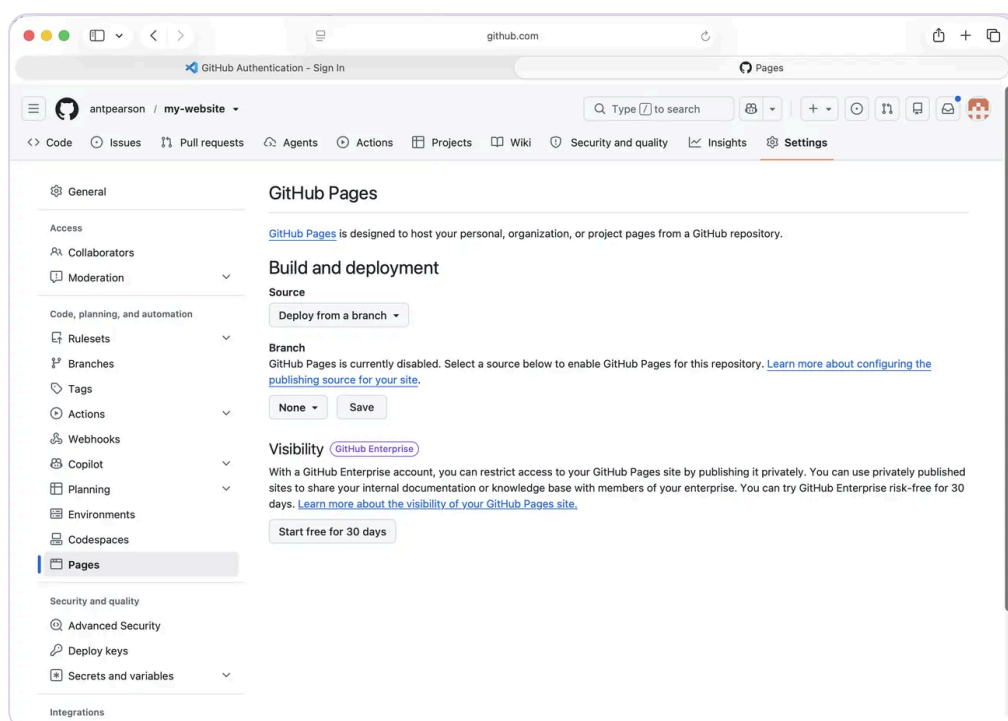
1. Save everything: **⌘S** in every tab with a dot.
2. Commit anything left under Changes, with a message.
3. Click **Sync Changes** until the button goes quiet.
4. Refresh your repository on github.com and see your last commit message.
5. Sign out of github.com in the browser before you log off the Mac.

Everything so far keeps your work safe. This part makes it public: GitHub Pages takes what is on your repository's **main** branch and serves it as a real website with its own address. You switch it on once, and from then on every commit you sync goes live on the internet a minute or two later, automatically. Nothing about your weekly routine changes.

Live means live. Everything in your repository is already publicly visible, and Pages puts it on a friendly web address anyone can visit. Coursework belongs here; passwords, private photos and anything you would not hand a stranger do not.

4.1 Find the Pages settings

1. On your repository page at github.com, click **Settings** at the right-hand end of the tab row.
2. In the left sidebar, click **Pages**.



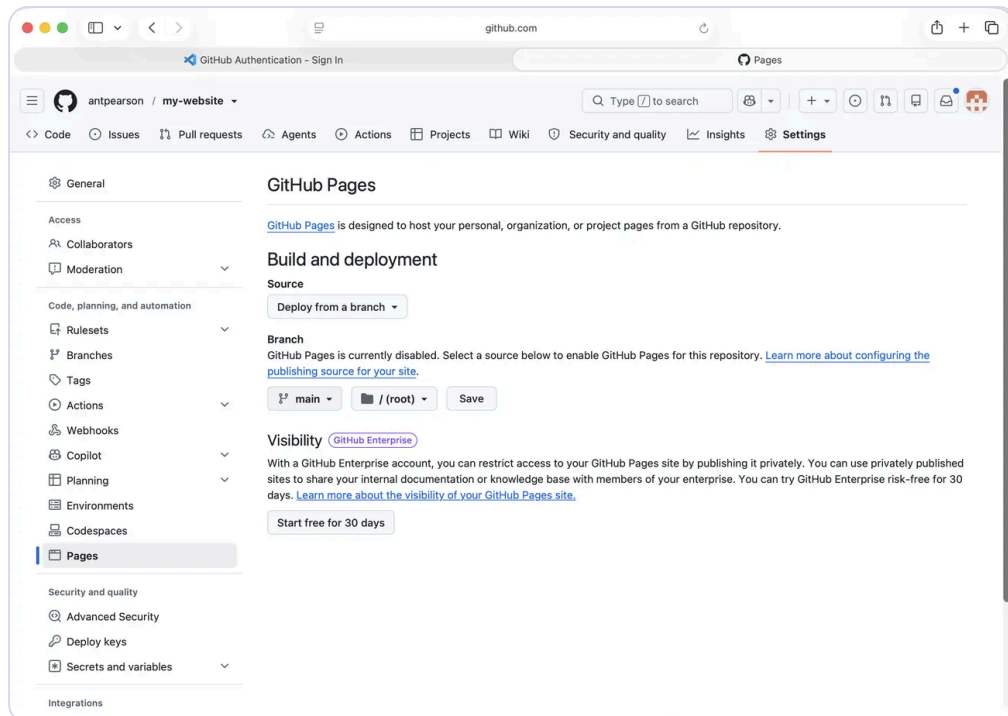
The Pages screen before it is switched on. Ignore the Visibility section and its free trial button: that is GitHub selling something you do not need.

4.2 Deploy from main

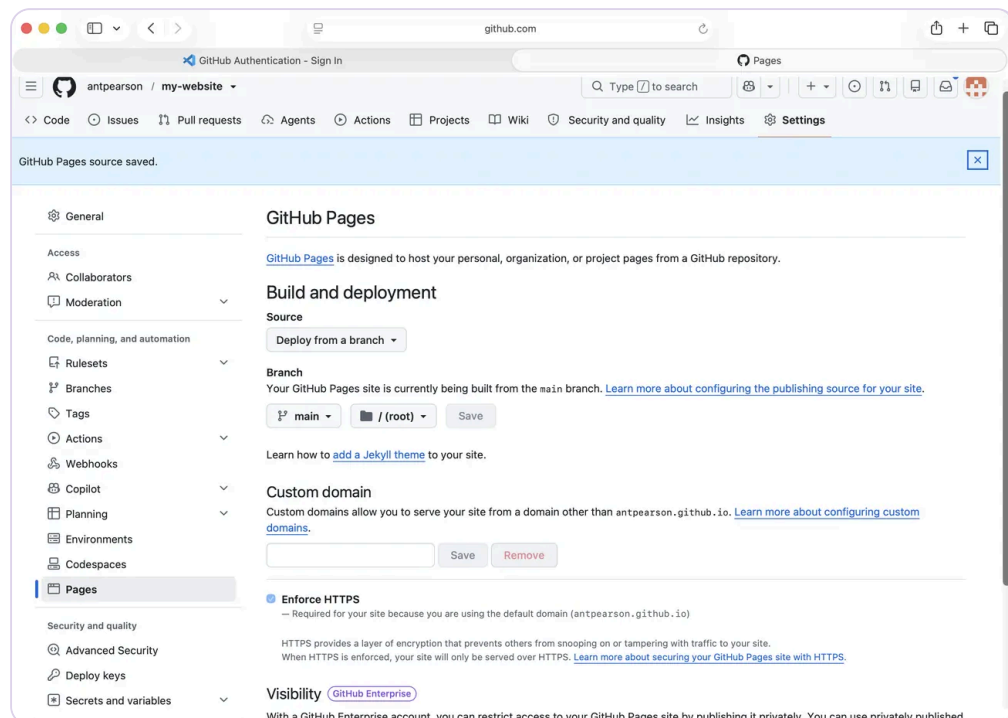
1. Under Build and deployment, set Source to **Deploy from a branch**.

2. Under Branch, choose **main** and leave the folder as **/ (root)**.

3. Click **Save**.



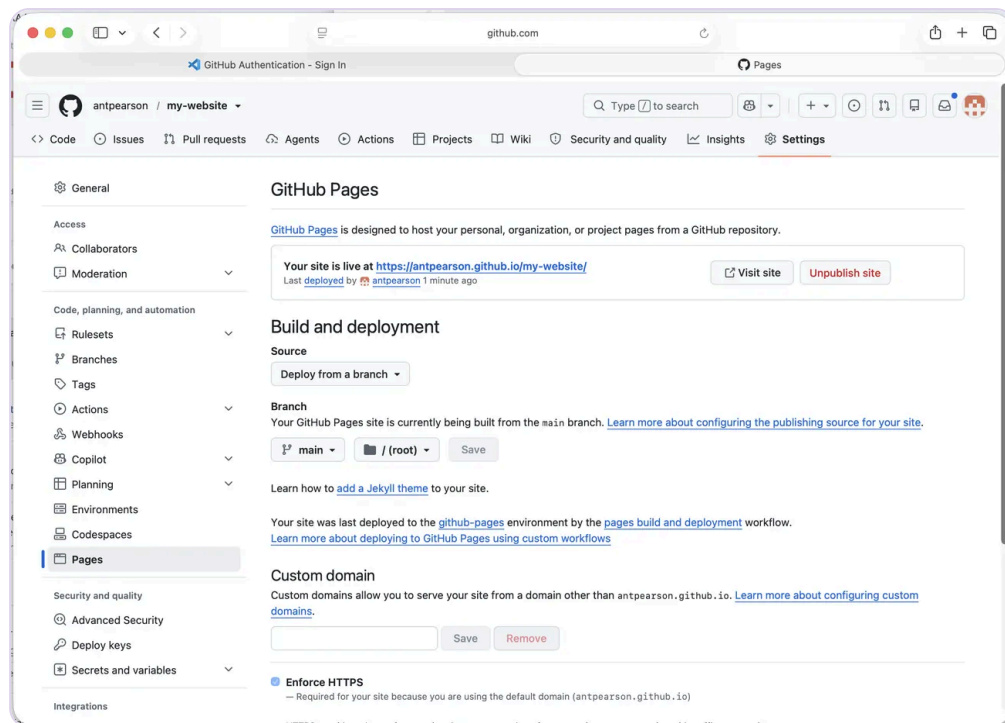
main, / (root), Save.



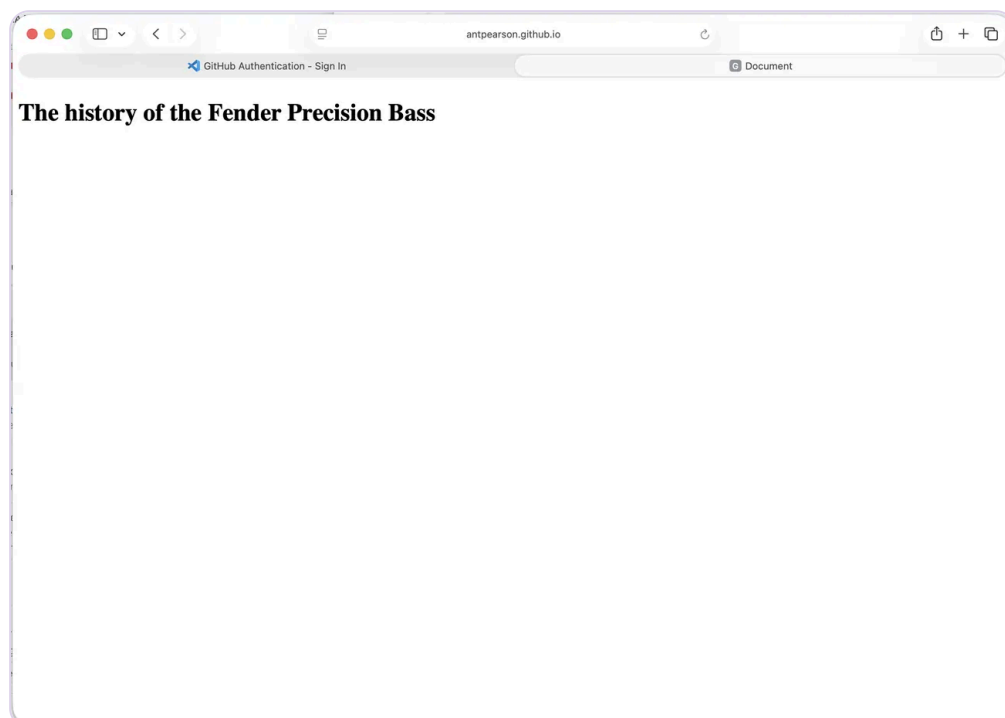
Saved. GitHub is now building your site behind the scenes.

4.3 Get your address

The build takes a minute or two. Refresh the Pages settings page until a box appears at the top saying **Your site is live at...** followed by your address. The shape is always the same: **`https://yourusername.github.io/my-website/`**. That works because your home page is called **`index.html`**, the one filename the web serves by default.



The address appears at the top once the first build finishes. Click **Visit site**.



Your page, on the internet, at an address you can put on your phone, a CV or a portfolio.

From now on, publishing is automatic. Commit and sync as in Part 3, wait a minute or two, refresh your live address, and the change is on the web. There is nothing extra to click, ever.

? When it goes wrong

- **"Make sure you configure your user.name and user.email"** when committing: you are on a machine that does not know you yet. Cancel, run the two lines from step 2.2, commit again.
- **A menu is greyed out:** a dialog box is open somewhere, possibly hidden behind the window. Find it, close it, try again.
- **You published as private by accident:** on github.com go to your repository's **Settings**, scroll to the Danger Zone, and choose **Change visibility** → **public**. Or ask a tutor.
- **Commit button says Sync Changes but you have edits:** your edits are not saved or not committed yet. Save, commit, then sync.
- **The browser sign-in page never comes back to VS Code:** close the browser tab, click **Publish Branch** (or retry the clone) and go through the sign-in again.
- **The live site is not showing your latest change:** give it two minutes, then refresh with  to skip the browser's saved copy. Still old? Check github.com shows the change; if it does not, you have not synced.